

MODULE DEVELOPMENT GUIDE

CyberSafe VR | XR Module Development Guide

Hardhat Enterprises

Deakin University Capstone

Last updated: May 2026

What This Guide Is For

This guide is for anyone on the team who wants to build a new cybersecurity module using the **XR_Module_Template** scene. It covers how everything in the template works, how to wire things up, and the common issues we ran into during development, so you don't have to figure them out yourself.

Read through this before you start building. It will save you a lot of time.

Contents

1. Project Structure Overview	4
2. Starting a New Module	5
3. Template Scene - Full Hierarchy Explained	6
4. SceneFader - Scene Transitions	8
5. LoadingText - The Loading Screen Message	10
6. ModuleCompleteCanvas - The End Screen	11
7. PauseMenu - The Pause System	13
8. XR UI Setup - Why It Works Differently in VR	15
9. Adding Your Scene to the Build Profile	17
10. Common Mistakes	18
Quick Reference.....	19

1. Project Structure Overview

Here is the relevant part of the project folder layout:

```
Assets/  
  Scenes/  
    MainMenu.unity  
    XR_Module_Template.unity    <- your starting point  
    SafeWebBrowsing01.unity  
    [other module scenes]  
  
  Prefabs/  
    XR_Rig_Template.prefab    <- drag into any scene for instant VR rig  
    XR_DeviceSimulator.prefab  <- drag in for desktop testing  
    ModuleCompleteCanvas.prefab <- reusable end screen  
  
  Scripts/  
    SceneFader.cs  
    PauseMenu.cs
```

2. Starting a New Module

Option A - Duplicate the Template (Recommended)

This is the easiest way. You get a fully working scene with everything already set up.

1. In Unity, go to Assets/Scenes/ in the Project panel
2. Right-click XR_Module_Template and select Duplicate
3. Rename the duplicate to your module name (e.g. PhishingModule)
4. Double-click to open it
5. Start building your content on top of it. The XR rig, fade transitions, pause menu and end screen are all already there

Option B - Fix an Existing Broken Scene

Use this if you are updating an older scene that has a broken or missing XR setup.

1. Go to Assets/Prefabs/
2. Drag XR_Rig_Template into the Hierarchy
3. Drag XR_DeviceSimulator into the Hierarchy
4. Delete any old broken XR Origin that was already in the scene
5. Make sure EventSystem in the scene has XR UI Input Module (see Section 8)
6. Press Play to test

***Note:** After you are done, don't forget to add your scene to the Build Profile (see Section 9).*

3. Template Scene - Full Hierarchy Explained

When you open XR_Module_Template, the Hierarchy looks like this:

```
XR_Module_Template
  XR Origin (XR Rig)
    Camera Offset
    Main Camera
    Left Controller
      Near-Far Interactor    <- handles VR ray and UI interaction
      Poke Interactor
      Teleport Interactor
      WristCanvas           <- pause button attached to left wrist
        PauseButton
    Right Controller
      Near-Far Interactor
      Poke Interactor
      Teleport Interactor
    Gaze Interactor
    Locomotion
      Move Provider
      Snap Turn Provider
      Teleportation Provider
      Climb Provider

  XR Device Simulator    <- desktop testing (disable on real headset)

  FadeCanvas             <- Screen Space Overlay, Sort Order 10
  FadeImage              <- full-screen black overlay
  LoadingText            <- shown during scene transitions

  SceneFader             <- empty GameObject, holds SceneFader.cs

  Menu                   <- World Space Canvas (your module UI here)
    PlaceholderButton    <- replace with your own buttons

  ModuleCompleteCanvas   <- World Space Canvas (DISABLED by default)
  WellDoneText
```

ReturnToMenuButton

PauseManager <- empty GameObject, holds PauseMenu.cs

PauseMenuCanvas <- World Space Canvas (DISABLED by default)

Panel

ResumeButton

RestartButton

ReturnToMenuButton

Two canvases are disabled by default: ModuleCompleteCanvas and PauseMenuCanvas. This is intentional. Both are already fully set up and wired, they just shouldn't be visible when the scene first loads. Each section below explains when and how to activate them.

4. SceneFader - Scene Transitions

What It Does

SceneFader handles smooth fade-to-black transitions between scenes. Instead of scenes cutting abruptly, the screen fades to black, the next scene loads, then fades back in when it's ready.

How to Call It From Your Script

```
// Find the SceneFader in the scene
SceneFader fader = FindObjectOfType<SceneFader>();

// Call FadeToScene and pass in the target scene name
fader.FadeToScene("MainMenu");
```

Or wire it directly from a button's **On Click ()** in the Inspector:

- Drag SceneFader from the Hierarchy into the object field
- Function: SceneFader -> FadeToScene (string)
- In the text box that appears, type the exact scene name (e.g. MainMenu)

The scene name must match the filename exactly, including capitalisation. **MainMenu** is not the same as **mainmenu**.

How the Fade Actually Works

```
FadeToScene("MainMenu") is called
|
FadeOutAndLoad coroutine runs
|
Screen fades from transparent to black
|
LoadingText becomes visible ("Loading...")
|
SceneManager.LoadScene("MainMenu") fires
|
New scene loads
|
FadeIn coroutine runs automatically on Start()
|
Screen fades from black back to transparent
```

Inspector Setup (Already Done in Template)

The **SceneFader** GameObject has SceneFader.cs attached. Two fields need to be linked:

<i>Field</i>	<i>What to Link</i>
Fade Image	FadeCanvas -> FadeImage
Loading Text	FadeCanvas -> LoadingText

Both are already linked in the template. If you are setting this up from scratch in a new scene, drag those objects into the fields manually.

FadeCanvas Settings

- **Render Mode:** Screen Space - Overlay
- **Sort Order:** 10 (ensures it draws on top of everything else in the scene)
- **FadeImage:** Full-screen Image, colour set to black. **Raycast Target must be unchecked** on this image, otherwise it will block button clicks when transparent.

5. LoadingText - The Loading Screen Message

What It Does

LoadingText is a TextMeshPro element inside FadeCanvas that shows "Loading..." on the black screen between scenes so players know something is happening.

How It Works

- On Start(), SceneFader.cs hides LoadingText automatically
- When FadeOutAndLoad() runs and the screen is fully black, LoadingText becomes visible
- When the next scene loads, it starts fresh and hides it again on Start()

Why We Hide It in Code and Not in the Editor

Another thing to keep in mind is that, why not just uncheck LoadingText in the Hierarchy to hide it. But if you disable the object in the editor, SceneFader loses the reference to it and can't re-enable it when it needs to. Hiding it through code (keeping the object active but invisible) keeps the reference working correctly.

***Note:** Do not disable LoadingText in the Hierarchy. Leave it enabled and let the script manage it.*

6. ModuleCompleteCanvas - The End Screen

What It Does

ModuleCompleteCanvas is a World Space Canvas that shows a "Well Done!" message and a Return to Main Menu button when a player finishes a module. It is disabled by default. You activate it from your module script when the player has completed whatever your module requires.

How to Activate It

```
// Assign this in the Inspector by dragging ModuleCompleteCanvas in
public GameObject moduleCompleteCanvas;

// Call this when the player finishes the module
public void OnModuleComplete()
{
    moduleCompleteCanvas.SetActive(true);
}
```

What Is Already Set Up

You don't need to wire the button yourself. It's already done:

- WellDoneText - TextMeshPro message, change the text if you want
- ReturnToMenuButton - On Click() is already wired to
SceneFader.FadeToScene("MainMenu")

Just activate the canvas and everything works.

Using the Prefab in a New Scene

ModuleCompleteCanvas is saved as a prefab in Assets/Prefabs/. To use it in a new scene:

1. Drag ModuleCompleteCanvas from Assets/Prefabs/ into the Hierarchy
2. Make sure it is disabled (uncheck the checkbox at the top of the Inspector)
3. Assign the Event Camera field to Main Camera (see Section 8)
4. In your module script, reference it and call SetActive(true) when the player finishes

Canvas Settings

<i>Setting</i>	<i>Value</i>
Render Mode	World Space
Event Camera	Main Camera
Scale	0.005 on all axes
Graphic Raycaster	Tracked Device Graphic Raycaster

7. PauseMenu - The Pause System

What It Does

The pause system lets players pause a module at any time. In VR they press the button on the left wrist. On desktop they press P on the keyboard. A panel appears in front of the player's camera with Resume, Restart, and Return to Menu options.

Hierarchy Structure

Left Controller	
WristCanvas	<- always visible on the left wrist
PauseButton	<- opens the pause menu
PauseManager	<- empty GameObject at the scene root PauseMenu.cs lives here
PauseMenuCanvas	<- DISABLED by default
Panel	
ResumeButton	
RestartButton	
ReturnToMenuButton	

Why PauseManager Is a Separate Object

This is worth understanding. If the PauseMenu script lived on PauseMenuCanvas, it would be disabled whenever the canvas is disabled. A disabled script can't receive input or run any code. By putting the script on PauseManager (which is always active at the scene root), the script always runs regardless of whether the canvas is visible.

Button Wiring

<i>Button</i>	<i>Object to drag in</i>	<i>Function</i>
PauseButton (on WristCanvas)	PauseMenuCanvas	PauseMenu -> TogglePause
ResumeButton	PauseMenuCanvas	PauseMenu -> ResumeGame
RestartButton	PauseMenuCanvas	PauseMenu -> RestartScene
ReturnToMenuButton	SceneFader	SceneFader -> FadeToScene -> MainMenu

Keyboard Shortcuts for Desktop Testing

<i>Key</i>	<i>Action</i>
P	Pause
R	Resume

These only work when testing on desktop with the XR Device Simulator. They can be removed before a production build if needed.

PauseMenuCanvas Settings

<i>Setting</i>	<i>Value</i>
Render Mode	World Space
Event Camera	Main Camera
Scale	0.001 on all axes
Graphic Raycaster	Tracked Device Graphic Raycaster

The panel automatically positions itself in front of the player's camera when it appears, using `Camera.main.transform` inside `PauseMenu.cs`.

8. XR UI Setup - Why It Works Differently in VR

This section explains the reasoning behind the XR setup. Understanding this means you can debug issues yourself instead of guessing.

Why Tracked Device Graphic Raycaster?

Normal Unity UI uses a Graphic Raycaster to detect when the mouse or a touch hits a button. In VR there's no mouse. Interaction comes from the XR ray (the laser pointer from the controller). The default Graphic Raycaster has no idea what an XR ray is and ignores it completely.

Tracked Device Graphic Raycaster is the XR version. It understands the XR input system and responds to ray interactions from the controllers.

Any World Space Canvas that needs to be interactive in VR must use Tracked Device Graphic Raycaster. If you create a new canvas, delete the default Graphic Raycaster component and add Tracked Device Graphic Raycaster in its place.

Why Does the Event Camera Need to Be Assigned?

In Screen Space mode, Unity figures out which camera to use automatically. In World Space mode it doesn't, so you have to tell it manually. Without an Event Camera assigned, the canvas can't process any UI events at all and buttons will never respond.

Every World Space Canvas needs Main Camera dragged into the Event Camera field in the Inspector. This applies to ModuleCompleteCanvas, PauseMenuCanvas, and any canvases you create yourself.

Why XR UI Input Module on the EventSystem?

The EventSystem drives all UI interaction in Unity. By default it uses Standalone Input Module which handles mouse and keyboard input. In XR you need XR UI Input Module instead because it translates controller actions into UI events that the canvas system can understand.

If you create a new scene from scratch rather than duplicating the template, you need to:

1. Click on the EventSystem in the Hierarchy
2. Remove Standalone Input Module
3. Add XR UI Input Module

The template already has this set up correctly.

Near-Far Interactor and Cast Distance

The Near-Far Interactor is what fires the ray from the controller in XR Toolkit 3.x. It replaced the old Ray Interactor with previous versions. Do not add a Ray Interactor alongside it, having both causes console warnings and input conflicts.

The cast distance controls how far the ray reaches. During development the working value we landed on is 20. If you ever see a ray that stops short of a button, this is the first thing to check:

1. In the Hierarchy, expand Left Controller (or Right Controller)
2. Click on Near-Far Interactor
3. In the Inspector find Curve Casting Settings -> Cast Distance
4. Set it to 20

Do this for both Left and Right Controller.

9. Adding Your Scene to the Build Profile

Unity can only load scenes it knows about. If your scene isn't registered, `SceneManager.LoadScene()` will throw an error or do nothing. Every time you create a new module scene, add it to the Build Profile.

1. Go to File - Build Profiles
2. Find the Scenes In Build list
3. Navigate to Assets/Scenes/ in the Project panel
4. Drag your new scene into the list
5. Check that MainMenu is already in there

Each scene in the list gets a number on the right. This is its build index. The order doesn't matter much, except index 0 should be MainMenu since that's the first thing that loads.

10. Common Mistakes

ModuleCompleteCanvas or PauseMenuCanvas is visible on scene start

Both should be disabled by default. Click on the canvas in the Hierarchy and uncheck the checkbox at the very top of the Inspector.

Buttons don't respond to the ray

Work through this list in order:

1. Is the canvas set to World Space?
2. Is Event Camera assigned (should be Main Camera)?
3. Is the Graphic Raycaster the Tracked Device version?
4. Does the EventSystem have XR UI Input Module?
5. Is the Near-Far Interactor cast distance set to 20 on both controllers?
6. Is Raycast Target checked on the button's Image component?

Scene won't load when FadeToScene is called

The scene isn't in the Build Profile. Go to File -> Build Profiles and drag it in.

PauseMenu script doesn't respond to P key

The script is probably on PauseMenuCanvas which is disabled. Move PauseMenu.cs to the PauseManager GameObject instead.

XR Device Simulator isn't working on desktop

Make sure the XR Device Simulator object is in the Hierarchy and its GameObject is enabled. It needs to be active to do anything.

FadeImage is blocking button clicks when the screen looks clear

Raycast Target is checked on FadeImage. Click FadeImage, find the Image component in the Inspector, and uncheck Raycast Target.

Quick Reference

<i>I want to...</i>	<i>How to do it</i>
Start a new module	Duplicate XR_Module_Template scene
Fix an old broken scene	Drag XR_Rig_Template and XR_DeviceSimulator prefabs in
Transition to another scene	SceneFader.FadeToScene("SceneName")
Show the module complete screen	moduleCompleteCanvas.SetActive(true)
Test on desktop without a headset	Make sure XR_DeviceSimulator is in the Hierarchy and enabled
Register a new scene for loading	Add it to File -> Build Profiles
Debug a button that won't click	Check: World Space, Event Camera, Tracked Device Raycaster, XR UI Input Module, cast distance 20

For questions check the Unity XR Interaction Toolkit 3.0 documentation.